Studio Nx

백엔드 236 — 연결선

2026-10-18

Backend(FastAPI/Python) 236 — `track_service` 연결선 (갈래 나누기와 합치기)

항목	내용
파일	`studionx/backend/app/services/track_service.py`
작업	S1-09 기본 악기·이펙트: 이펙트 연결선 읽기·긋기·지우기 (PRD 02 FR-265, FR-267, FR-269)
작성	Claude (AI), 2026-10-18, 사용자 검토
이전 · 다음	235 이펙트 칸 · 237 플러그인 API

1. 하는 일

갈래(패처) 구성에서 이펙트 카드를 잇는 선(`track_plugin_links`)을 다루는 함수 세 개를 둡니다.

함수	하는 일	권한
`list_links`	트랙의 연결선 목록 (번호 순)	멤버면 누구나
`add_link`	연결선 한 줄 긋기. 가리키는 이펙트가 같은 트랙의 것인지 확인	OWNER, EDITOR
`remove_link`	연결선 지우기	OWNER, EDITOR

`from` 하나에서 여러 `to` 로 가면 갈래가 나뉘고, 여러 `from` 이 `to` 하나로 모이면 병합입니다. `from` 이 비어 있으면 트랙 입력(소스), `to` 가 비어 있으면 트랙 출력입니다 (FR-267).

2. 언제 쓰나

쓰는 곳	
`GET/POST /tracks/{id}/links`, `DELETE /tracks/{id}/links/{lid}` (241)	세 라우터가 이 함수들을 부릅니다
화면의 갈래(패처) 보드	카드를 이어 갈래를 만들고, 병합하고, 선을 지울 때 부릅니다 (FR-265)
S1-10 저장·불러오기	갈래 구성을 함께 저장하고 되살립니다 (FR-254)

3. 코드

목록

```
def list_links(db: Session, user: User, track_id: int) -> Sequence[TrackPluginLink]:
    """트랙의 연결선 목록 (FR-267). 멤버면 누구나."""
    track = require_track(db, user, track_id)
    stmt = (
        select(TrackPluginLink)
        .where(TrackPluginLink.track_id == track.track_id)
        .order_by(TrackPluginLink.link_id)
    )
    return db.execute(stmt).scalars().all()
```

긋기

```
def add_link(db, user, track_id, *, from_track_plugin_id=None, from_port=0,
             to_track_plugin_id=None, to_port=0, gain_db=Decimal("0.00")):
    """갈래 연결선 붙이기 (FR-267·269). OWNER·EDITOR 만."""
    track = _require_editor_track(db, user, track_id)
    for plugin_id in (from_track_plugin_id, to_track_plugin_id):
        if plugin_id is None:
            continue
        tp = db.get(TrackPlugin, plugin_id)
        if tp is None or tp.track_id != track.track_id:
            raise AppError(ErrorCode.NOT_FOUND, 404, "Track effect not found.")
    link = TrackPluginLink(track_id=track.track_id, from_track_plugin_id=from_track_plugin_id,
                           from_port=from_port, to_track_plugin_id=to_track_plugin_id,
                           to_port=to_port, gain_db=gain_db)
    db.add(link)
    db.commit()
    db.refresh(link)
    return link
```

지우기

```
def remove_link(db: Session, user: User, track_id: int, link_id: int) -> None:
    """연결선 떼기 (FR-267). OWNER·EDITOR 만."""
    link = db.get(TrackPluginLink, link_id)
    if link is None or link.track_id != track_id:
        raise AppError(ErrorCode.NOT_FOUND, 404, "Track plugin link not found.")
    _require_editor_track(db, user, track_id)
    db.delete(link)
    db.commit()
```

문법 풀이

식	뜻
`for plugin_id in (from, to)`	두 끝을 같은 코드로 검사합니다. 이펙트가 없거나 남의 트랙 것이면 404 입니다
`if plugin_id is None: continue`	트랙 입력이나 출력(비어 있는 끝)은 검사할 이펙트가 없습니다
`db.get(TrackPlugin, plugin_id)`	이펙트 칸이 실제 있는지 봅니다
`tp.track_id != track.track_id`	다른 트랙의 이펙트를 가리키는 선은 404 입니다. 갈래는 트랙 안에서만 이뤄집니다
`external` 이름 대신 `from·to`	`from` 이 파이썬 예약어라 인자 이름에 쓸 수 없습니다. 그래서 `from_track_plugin_id` 처럼 표의 칸 이름을 그대로 씁니다
`.order_by(link_id)`	만든 순서대로 돌려줍니다. 화면이 선을 늘 같은 차례로 그립니다

4. 왜 이렇게

선택	이유
선을 지우고 다시 긋는 방식	지금 화면에는 선 하나를 고치는 동작이 없습니다. 필요해지면 `PATCH` 를 더합니다 (`TrackPluginLinkUpdateRequest` 는 이미 준비되어 있습니다)
양 끝이 한 트랙 안	트랙 사이의 연결은 다른 표(`track_connections`, DB.md Phase 2)가 맡습니다. 이 표는 트랙 안 갈래 전용입니다 (FR-266)
빈 끝을 허용	트랙 입력과 출력도 같은 표로 담아야 화면이 선을 다 그릴 수 있습니다 (FR-267)
이펙트를 지우면 선도 삭제	DB 의 `ON DELETE CASCADE` 가 처리합니다. 없는 카드를 가리키는 선이 남지 않습니다
`gain_db` 를 선에 둠	갈래마다 세기를 다르게 보낼 수 있습니다 (FR-266)

5. 시험

`tests/test_track_routes.py` 의 `test_patcher_links_crud` 가 트랙 입력에서 이펙트 A 로 가는 선, 이펙트 A 에서 이펙트 B 로 가는 선을 만들고, 목록 개수와 `gain_db` 저장, 지우기를 확인합니다. 다른 트랙의 이펙트를 가리키면 404 입니다.